iRAYPLE Acquisition Card

SDK Development Document (C# Interface)



V1.3.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	Nov. 2022	1.0.0	First Release.	2.4.0
2	Oct. 2023	1.1.0	Added the Interface of Notification Channel.	2.4.1
3	Apr. 2023	1.2.0	Added Save Image Function.	2.4.2
4	Nov. 2025	1.3.0	Added MVPImage Construction Method	2.6.1

Table of Contents

1 Overview 4

1.1 Compatibility Introduction 4

1.2 SDK Installation Directory 4

1.2.1 SDK for Windows 4

1.3 Related Resources of SDK (C#) 5

1.4 SDK Engineering Configuration 6

1.4.1 VS2010~VS2022 6

2 Overall Calling Process of SDK 9

3 Function Interface Calling of SDK 10

3.1 System Operation Instruction 10

3.1.1 Acquisition Card Enumeration (Find Device) 10

3.1.2 Camera Enumeration (Find Device) 10

3.1.3 Obtain Version Information 11

3.2 Acquisition Card Operation 11

3.2.1 Open and Close Acquisition Card 11

3.2.2 Register Event Callback 12

3.3 Camera Operation Instruction 12

3.3.1 Camera Open and Close 12

3.3.2 Register Event Callback 13

3.4 Camera Operation Instruction 14

3.4.1 Stream Data Configuration 14

3.4.2 Image Collection 14

3.4.3 Acquire ImageData 15

3.4.4 Operations on Streaming and Image Data 17

3.5 Attribute Operation Instruction 18

3.5.1 Configuration Download, Storage and Loading 18

3.5.2 Attribute Determination, Acquisition and Setting 19

3.5.3 Execution of Command Attribute 25

3.5.4 Attribute Search and Usage 26

3.6 Image Operation 35

3.6.1 Video Recording 35

3.6.2 Image Operation 35

4 Image Format of 3rd-party Platform Conversion 38

4.1 Halcon Object (HObject) 38

4.1.1 Mono8 (Monochrome Image) 38

4.1.2 Non-Mono8 (Color Image) 38

4.2 MvpImage Object 39

4.2.1 Mono8 (Monochrome Image) 39

4.2.2 Non-Mono8 (Color Image) 39

5 Matched Routine Description 40

1 Overview

1.1 Compatibility Introduction

Table 1-1 Operating System Requirement

OS Type	Version	Note
Windows	Win7 SP1 32/64bit	Ensure that OS version includes the patch of SP1.
	Win8 32/64bit	
	Win10 32/64bit	

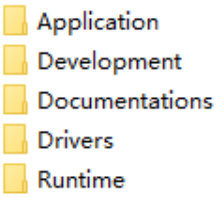
Windows XP is no longer supported since SDK V2.2.5.

1.2 SDK Installation Directory

1.2.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-1 Installation directory structure



Since the V2.1.0 of SDK, user can select the installation path.

Table 1-2 Installation directory structure description

File	Description
Application	● **MViewer** is compatible with 32-bit and 64-bit. ● **CamTools** is compatible with 32-bit and 64-bit.
Development	Stored various resource files required for the secondary development of the SDK. ● SDK header file, Lib file ● dotNet SDK library

	● Third-party plug-in libraries (Halcon and Sherlock). ● SDK Routine (C、MFC、C#、VB.Net、QT、Delphi、Python).
Documentations	● .**Net** interface document ● **ActiveX** interface document ● **C** interface document ● **SDK** development document
Drivers	● The installation and uninstallation software of **GigE** driver. ● The installation and uninstallation software of **USB** driver. ● The installation and uninstallation software of **virtual serial port** driver. ● The installation and uninstallation software of **DShowFilter**. ● The installation and uninstallation software of **PCIE** driver.
Runtime	SDK runtime library (including **GenICam** V3.0 third-party library). It contains 32-bit to 64-bit

1.3 Related Resources of SDK (C#)

Table 1-3 SDK source description

Category	Item	File Storage Folder	Name
SDK Library	DOTNET_3.5 (32-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMVFG\DOTNET_3.5\x86	MVFGSDK_Net.dll
	DOTNET_3.5 (64-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMVFG\DOTNET_3.5\x64	MVFGSDK_Net.dll
	DOTNET_4.0 (32-bit)	【SDK Installation Directory】\Development\DOTNET DLL\IMVFG\DOTNET_4.0\x86	MVFGSDK_Net.dll
	DOTNET_4.0 (64-bit)	【SDK Installation Directory】\Development\DOTNET DLL\	MVFGSDK_Net.dll

		IMVFG\DOTNET_4.0\x64	
Development Document	Chinese	【SDK Installation Directory】\ Documentations\	IMVFG_API_Net_CHS.chm Acquisition Card SDK Development Document (C#)
	English	【SDK Installation Directory】\ Documentations\	IMVFG_API_Net_ENG.chm
Routine	—	【SDK Installation Directory】 \Development\Samples\C#\IMVFG	

1.4 SDK Engineering Configuration

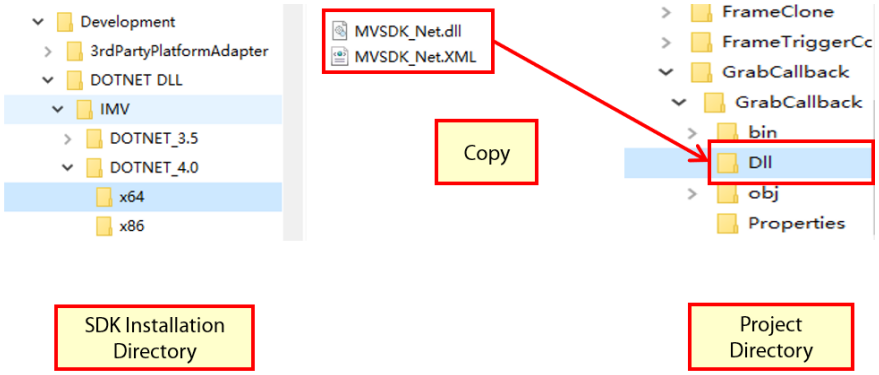
1.4.1 VS2010~VS2022

Take the project configuration of DOTNET4.0 64-bit as the example to illustrate the project configuration method.

Procedure

Step 1 Copy the related sources of SDK into the project directory.

Figure 1-2 Project configuration



Step 2 Click Project > Add the reference of the **MVFGSDK_Net.dll**.

Figure 1-3 Add reference

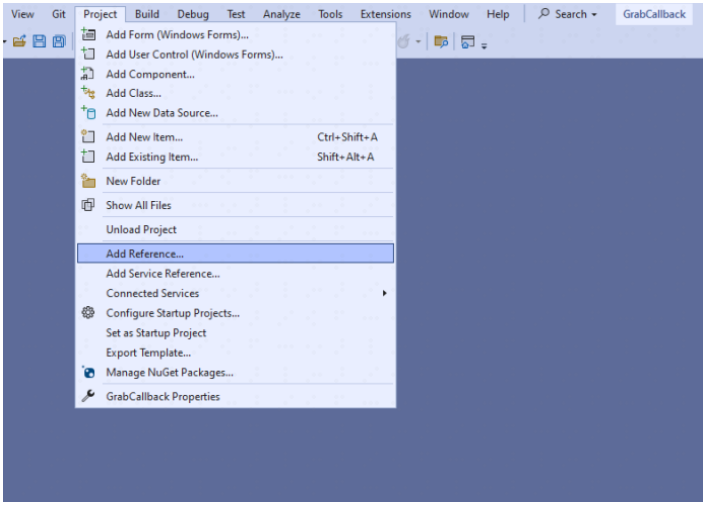
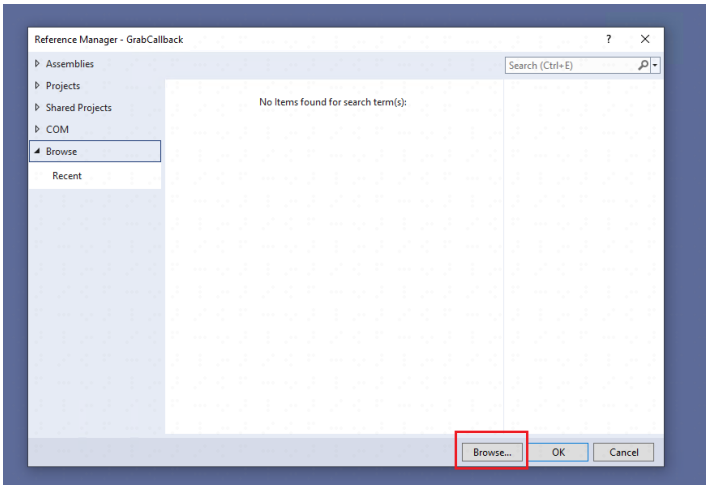
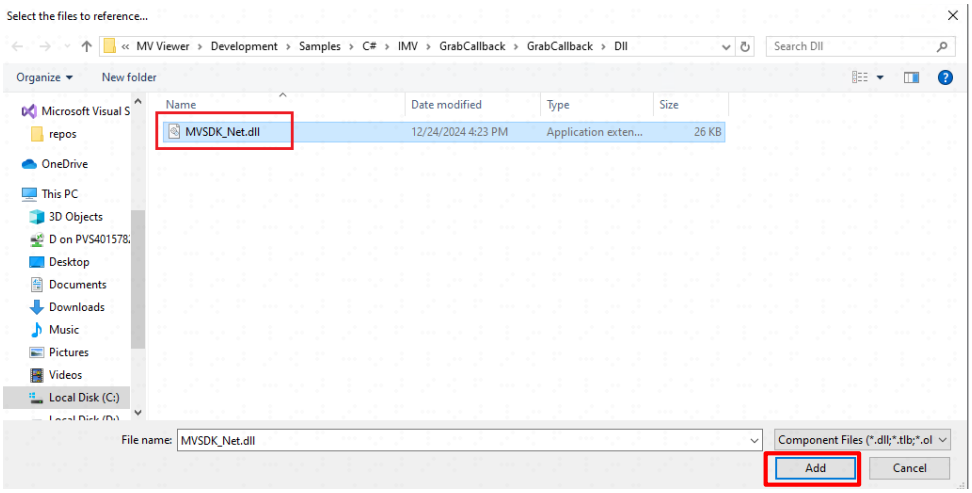
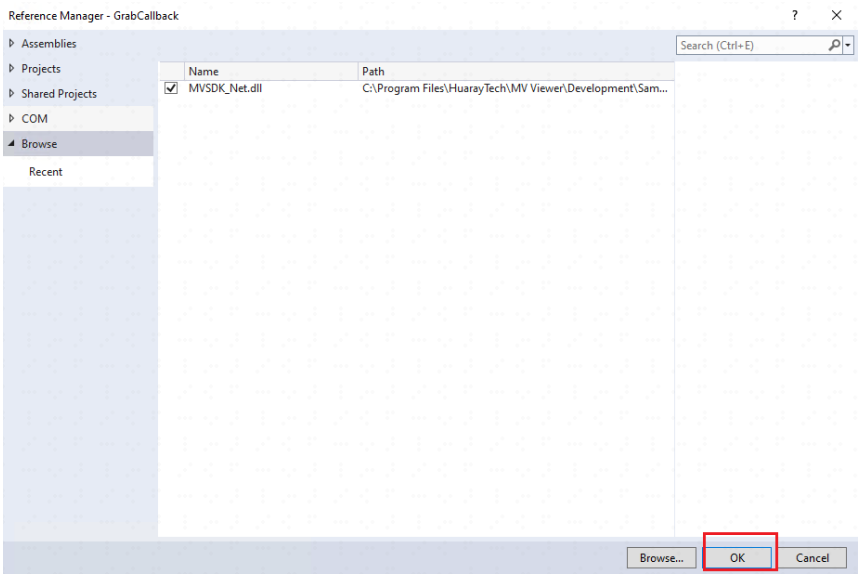


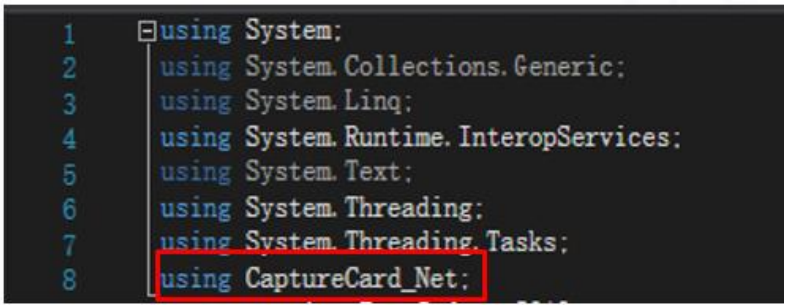
Figure 1-4 Add project file





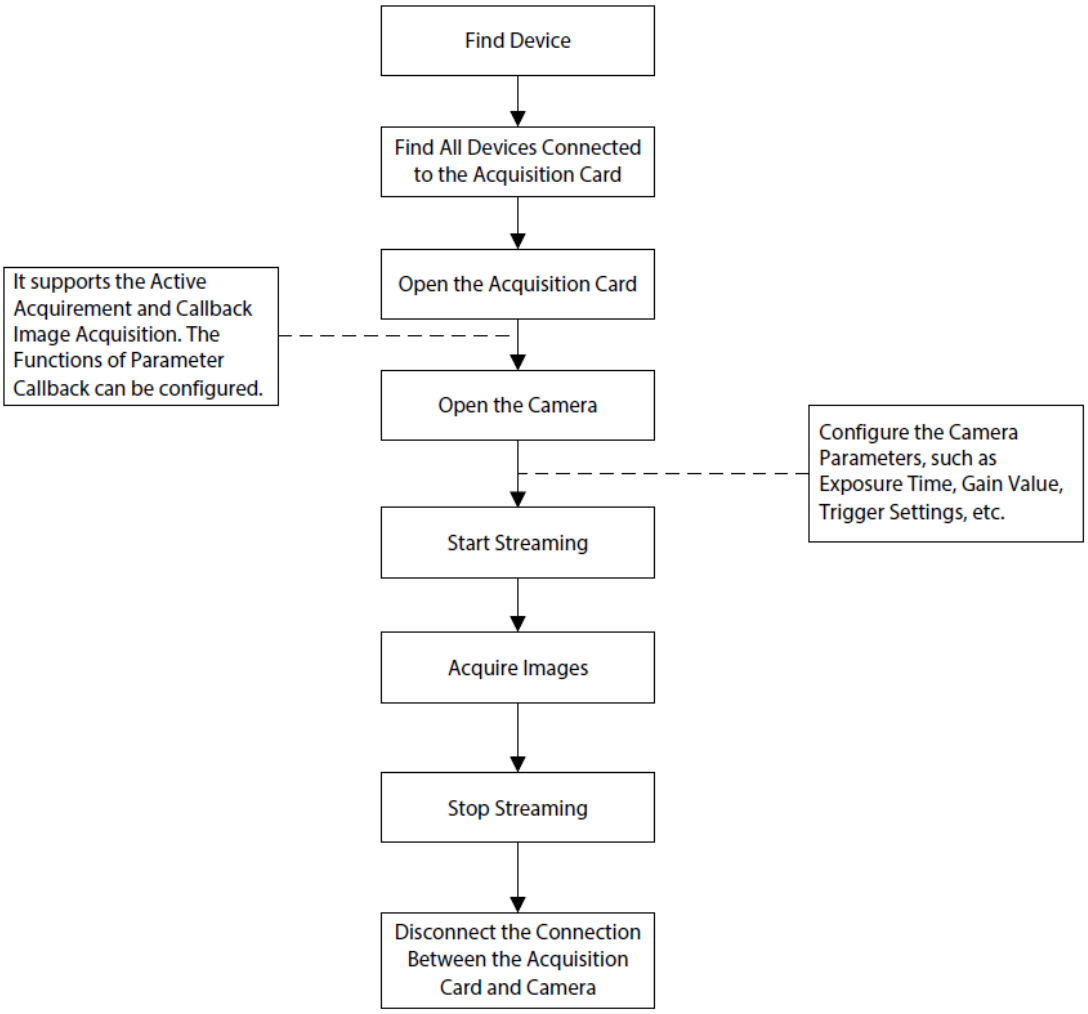


Step 3 Add "using CaptureCard_Net" in the calling code.



2 Overall Calling Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of SDK

3.1 System Operation Instruction

3.1.1 Acquisition Card Enumeration (Find Device)

- Interface Class: CardDev
- Interface: public static int IMV_FG_EnumInterface(uint interfaceType, ref IMVFGDefine.IMV_FG_INTERFACE_INFO_LIST pInterfaceList);
  - ◇ Description: Acquisition card enumeration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-1Parameter description

Parameter	Description
pInterfaceList [OUT]	Device list.
interfaceType [IN]	The interface type to be enumerated can be combined arbitrarily, such as **typeGigEInterface | typeU3vInterface**.

  - □ When **interfaceType** = **typeGigEInterface**, enumerate all online devices connected through the **GigE** interface.
  - □ When **interfaceType** = **typeU3vInterface**, enumerate all online devices connected through the **USB** interface.
  - □ When **interfaceType** = **typeCLInterface**, enumerate the online devices connected through the **CameraLink** interface.
  - □ When **interfaceType** = **typeCXPInterface**, enumerate all online devices connected through the **CXP** interface.
  - □ The **interfaceType** of the interface support any combination of interface types. For example, if you need to enumerate all online devices connected through the GigE port and USB 3.0 port, the **interfaceType** can be written as the **interfaceType** = **typeGigEInterface | typeU3vInterface**.
  - ◇ Reference Routine: All current routines can be referenced.

3.1.2 Camera Enumeration (Find Device)

- Interface Class: CamDev

- Interface: public static int IMV_FG_EnumDevice(uint nInterfaceType, ref IMVFGDefine.IMV_FG_DEVICE_INFO_LIST pDeviceList);
  - ◇ Description: Enumeration Device. Enumerate the cameras connected to the opened acquisition card.

Table 3-2 Parameter description

Parameter	Description
nInterfaceType [IN]	Enumeration device interface type.
pDeviceList [OUT]	Device list.

  - ◇ Reference Routine: All current routines can be referenced.

3.1.3 Obtain Version Information

- Interface Class: CardDev
- Interface: public static string IMV_FG_GetVersion();
  - ◇ Description: Get OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: **CommPropAccess**.

3.2 Acquisition Card Operation

3.2.1 Open and Close Acquisition Card

- Interface Class: CardDev
- Interface: public int IMV_FG_OpenInterface(uint nIndex);
  - ◇ Description: Open acquisition card. If succeeded, return **IMV_FG_OK**. If failed, return error code.
  - ◇ Reference Routine: All current routines can be referenced.
- Interface Class: CardDev
- Interface: public int IMV_FG_IsOpenInterface();
  - ◇ Description: Determine whether the acquisition card is opened. If opened, return **True**. If close or offline, return **False**.
  - ◇ Reference Routine: All current routines can be referenced.
- Interface Class: CardDev
- Interface: public int IMV_FG_CloseInterface();
  - ◇ Description: Close acquisition card. If succeeded, return **IMV_FG_OK**. If failed,

return error code.

◇ Reference Routine: All current routines can be referenced.

3.2.2 Register Event Callback

- Interface Class: CardDev
- Interface: public int IMV_FG_SubscribeMsgChannelArg(IMVFGDefine.IMV_FG_MsgChannelCallBack proc, IntPtr pUser);
  - ◇ Description: Notification channel event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-3 Parameter description

Parameter	Description
proc [IN]	Device notification channel event callback function.
pUser [IN]	User-defined data, can be set to **NULL**.

Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

  - ◇ Reference Routine: **MsgEvents**.

3.3 Camera Operation Instruction

3.3.1 Camera Open and Close

- Interface Class: CamDev
- Interface: public int IMV_FG_OpenDevice(IMVFGDefine.IMV_FG_ECreateHandleMode mode, int cameraIndex = 0, string cameraStr = "");
  - ◇ Description: Open device. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-4 Parameter description

Parameter	Description
mode [IN]	Handle creation mode.
cameraIndex [IN]	Enable the camera identifier.
cameraStr [IN]	Open the string of the camera.

◇ Reference Routine: All current routines can be referenced.

- Interface Class: CamDev
- Interface: public int IMV_FG_OpenDeviceEx (IMVFGDefine.IMV_FG_ECreateHandleMode mode, int cameraIndex = 0, string cameraStr = "");
  ◇ Description: Open device (synchronize the parameters of acquisition card and camera). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-5 Parameter description

Parameter	Description
mode [IN]	Handle creation mode.
cameraIndex [IN]	Enable the camera identifier.
cameraStr [IN]	Open the string of the camera.

- Interface Class: CamDev
- Interface: public int IMV_FG_IsDeviceOpen();
  ◇ Description: Determine whether the device is opened. If opened, return **True**. If close or offline, return **False**.
- Interface Class: CamDev
- Interface: public int IMV_FG_CloseDevice();
  ◇ Description: Close device. If succeeded, return **IMV_FG_OK**. If failed, return error code.
  ◇ Reference Routine: All current routines can be referenced.

3.3.2 Register Event Callback

- Interface Class: CamDev
- Interface: public int IMV_FG_SubscribeConnectArg(IMVFGDefine.IMV_FG_ConnectCallBack proc, IntPtr pUser);
  ◇ Description: Camera connection status event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-6 Parameter description

Parameter	Description
proc [IN]	Device connection status event callback function.
pUser [IN]	User-defined data, can be set to **NULL**.



Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

3.4 Camera Operation Instruction

3.4.1 Stream Data Configuration

- Interface Class: CardDev
- Interface: public int IMV_FG_SetBufferCount(uint nSize);
  - ◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-7 Parameter description

Parameter	Description
Handle [IN]	Device handle.
nSize [IN]	Buffer quantity.

The configuration cannot be performed during the streaming.

3.4.2 Image Collection

- Interface Class: CardDev
- Interface: public int IMV_FG_StartGrabbing();
  - ◇ Description: Start acquiring images. If succeeded, return **IMV_FG_OK**. If failed, return error code.
  - ◇ Reference Routine: **Grab**.
- Interface Class: CardDev
- Interface: public int IMV_FG_StartGrabbingEx(UInt64 maxImagesGrabbed, IMVFGDefine.IMV_FG_EGrabStrategy strategy);
  - ◇ Description: Start acquiring images. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-8 Parameter description

Parameter	Description
maxImagesGrabbed [IN]	The max number of frames of acquired images. After the specified number of images is acquired,

	the acquisition will stop. If the specified number is 0, it means that the acquisition is continuously performed.  The parameter of **IMV_FG_StartGrabbing** is 0 by default.
strategy [IN]	Acquisition strategy (IMV_FG_StartGrabbing wil use the grabStrartegySequential to acquire image).

- ◇ Reference Routine: **GrabStrategy.**

- Interface Class: CardDev
- Interface: public int IMV_FG_IsGrabbing();
  - ◇ Description: Determine whether the device is acquiring images. If streaming, return True. If not streaming, return False.
- Interface Class: CardDev
- Interface: public int IMV_FG_StopGrabbing();
  - ◇ Description: Stop acquiring images. If succeeded, return **IMV_FG_OK**. If failed, return error code.
  - ◇ Reference Routine: **Grab**.

3.4.3 Acquire ImageData

- Interface Class: CardDev
- Interface: public int IMV_FG_AttachGrabbing(IMVFGDefine.IMV_FG_FrameCallBack proc, IntPtr pUser);
  - ◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-9 Parameter description

Parameter	Description
proc [IN]	Frame data information callback function. Do not handle the time-consuming operations in this function, otherwise, it will affect the real-time performance of subsequent frame data.
pUser [IN]	User-defined data, can be set to **NULL**.



- The asynchronous acquisition mechanism and synchronous acquisition mechanism (**IMV_FG_GetFrame**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.
- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **FrameClone**, **FrameTriggerCount**, **GrabCallback**.

- Interface Class: CardDev
- Interface: public int IMV_FG_GetFrame(ref IMVFGDefine.IMV_FG_Frame pFrame, uint timeoutMS);

◇ Description: Acquire a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-10 Parameter description

Parameter	Description
pFrame [OUT]	The frame data information.
timeoutMS[IN]	The timeout value for obtaining a frame of image. The **INFINITE** mode means that it will continuously wait until a frame of image is received, otherwise, it will stop acquiring (unit: ms).



This interface does not support the multi-thread callings.

The synchronous and asynchronous acquisition mechanism (**IMV_FG_AttachGrabbing**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.

Acquiring images through releasing the internal cache. Use **IMV_FG_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **Grab**.

- Interface Class: CardDev
- Interface: public int IMV_FG_ReleaseFrame(ref IMVFGDefine.IMV_FG_Frame Pframe);

◇ Description: Release the image buffer space. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-11 Parameter description

Parameter	Description
pFrame [IN]	The frame data information.

◇ Reference Routine: **Grab**.

3.4.4 Operations on Streaming and Image Data

- Interface Class: CardDev
- Interface: public int IMV_FG_CloneFrame(ref IMVFGDefine.IMV_FG_Frame pFrame, ref IMVFGDefine.IMV_FG_Frame pCloneFrame);
  - ◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-12 Parameter description

Parameter	Description
pFrame [IN]	Clone the source frame data information.
pCloneFrame [OUT]	New frame data information.

Use **IMV_FG_ReleaseFrame** to release the image buffer space.

  - ◇ Reference Routine: **FrameClone**.
- Interface Class: CardDev
- Interface: public int IMV_FG_GetStatisticsInfo(ref IMVFGDefine.IMV_FG_StreamStatisticsInfo pStreamStatsInfo);
  - ◇ Description: Obtain statistics information of streaming (after performing **IMV_FG_StartGrabbing** / **IMV_FG_StartGrabbingEx** to call the obtaining command). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-13 Parameter description

Parameter	Description
pStreamStatsInfo [OUT]	Stream statistics information.

  - ◇ Reference Routine: **StreamStatistics.**
- Interface Class: CardDev
- Interface: public int IMV_FG_ResetStatisticsInfo();
  - ◇ Description: Reset statistics information of streaming (after performing **IMV_FG_StartGrabbing** / **IMV_FG_StartGrabbingEx** to call the resetting command). If succeeded, return **IMV_FG_OK**. If failed, return error code.
  - ◇ Reference Routine: **StreamStatistics.**

3.5 Attribute Operation Instruction

3.5.1 Configuration Download, Storage and Loading

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_DownLoadGenICamXML(string pFullFileName);
  - ◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-14 Parameter description

Parameter	Description
pFullFileName[IN]	File saving path.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SaveDeviceCfg(string pFullFileName);
  - ◇ Description: Saving the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-15 Parameter description

Parameter	Description
pFullFileName [IN]	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.

  - ◇ Reference Routine: **LoadAndSaveDeviceCfg.**
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_LoadDeviceCfg(string pFullFileName, ref IMVFGDefine.IMV_FG_ErrorList pErrorList);
  - ◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-16 Parameter description

Parameter	Description
pFullFileName [IN]	Full name (including saving path) of the device configuration file (XML), such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList [OUT]	The list of attribute name that failed to load. The upper limit of attributes that fails to save and load is

	IMV_FG_MAX_ERROR_LIST_NUM. If succeeded, return **IMV_FG_OK**. If failed, return error code.

◇ Reference Routine: **LoadAndSaveDeviceCfg.**

3.5.2 Attribute Determination, Acquisition and Setting

3.5.2.1 Propriety Determination

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_FeatureIsAvailable(string pFeatureName);
  - ◇ Description: Determine whether the attribute is available. If available, return **True**. If unavailable, return **False**.

Table 3-17 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_FeatureIsReadable(string pFeatureName);
  - ◇ Description: Determine whether the attribute is readable. If readable, return **True**. If unreadable, return **False**.

Table 3-18 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_FeatureIsWriteable(string pFeatureName);
  - ◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

Table 3-19 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_FeatureIsStreamable(string pFeatureName);
  - ◇ Description: Determine whether the attribute is streamable. If streamable, return **True**. If unstreamable, return **False**.

Table 3-20 Parameter description

Parameter	Description

pFeatureName [IN]	Attribute name.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_FeatureIsValid(string pFeatureName);
  - ◇ Description: Determine whether the attribute is valid. If valid, return **True**. If invalid, return **False**.

Table 3-21 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: **Record**.

3.5.2.2 Attribute Acquisition

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetIntFeatureValue(string pFeatureName, ref Int64 pIntValue);
  - ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-22 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of Integer.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetIntFeatureMin(string pFeatureName, ref Int64 pIntValue);
  - ◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-23 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The min value that the integer can be configured.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetIntFeatureMax(string pFeatureName, ref Int64 pIntValue);
  - ◇ Description: Obtain the configurable max value of integer attribute. If succeeded,

return **IMV_FG_OK**. If failed, return error code.

Table 3-24 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the integer attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetIntFeatureInc(string pFeatureName, ref Int64 pIntValue);
  - ◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-25 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	Integer attribute step size.

◇ Reference Routine: **CommPropAccess**

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetDoubleFeatureValue(string pFeatureName, ref double pDoubleValue);
  - ◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-26 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pDoubleValue [OUT]	The attribute value of floating point.

◇ Reference Routine: **CommPropAccess.**

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetDoubleFeatureMin(string pFeatureName, ref double pDoubleValue);
  - ◇ Description: Obtain the configurable min value of the floating-point attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-27 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pDoubleValue [OUT]	The configurable min value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetDoubleFeatureMax(string pFeatureName, ref double pDoubleValue);
  ◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-28 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pDoubleValue [OUT]	The configurable max value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetBoolFeatureValue(string pFeatureName, ref bool pBoolValue);
  ◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-29 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pBoolValue [OUT]	The attribute value of Boolean.

◇ Reference Routine: **CommPropAccess**.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetStringFeatureValue(string pFeatureName, ref IMVFGDefine.IMV_FG_String pStringValue);
  ◇ Description: Obtain the string attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-30 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pStringValue [OUT]	The attribute value of String.

◇ Reference Routine: **CommPropAccess.**

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetEnumFeatureValue(string pFeatureName, ref UInt64 pEnumValue);
  ◇ Description: Obtain the string attribute value. If succeeded, return **IMV_FG_OK**.

If failed, return error code.

Table 3-31 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The attribute value of String.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetEnumFeatureSymbol(string pFeatureName, ref IMVFGDefine.IMV_FG_String pEnumSymbol);
  - ◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-32 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumSymbol [OUT]	Symbol value of attribute enumeration.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetEnumFeatureEntryNum(string pFeatureName, ref uint pEntryNum);
  - ◇ Description: Obtain the configurable number of the enumeration attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-33 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEntryNum [OUT]	The configurable quantity of the enumeration attribute.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_GetEnumFeatureEntrys(string pFeatureName, ref IMVFGDefine.IMV_FG_EnumEntryList pEnumEntryList);
  - ◇ Description: Obtain the enumeration value list of the configurable enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-34 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumEntryList [OUT]	The list of configurable enumeration value of enumeration attribute.

◇ Reference Routine: **CommPropAccess.**

3.5.2.3 Attribute Setting

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetIntFeatureValue(string pFeatureName, Int64 pIntValue);
  - ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-35 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pIntValue [IN]	The integer attribute value to be configured.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetDoubleFeatureValue(string pFeatureName, double pDoubleValue);
  - ◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-36 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pDoubleValue [IN]	Floating-point attribute value which is to be configured.

  - ◇ Reference Routine: **CommPropAccess.**
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetBoolFeatureValue(string pFeatureName, bool pBoolValue);
  - ◇ Description: Configure the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-37 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pBoolValue [IN]	Boolean attribute value which is to be configured.

  - ◇ Reference Routine: **CommPropAccess**.

- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetEnumFeatureValue(string pFeatureName, UInt64 pEnumValue);
  - ◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-38 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumValue [IN]	Enumeration attribute value which is to be configured.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetEnumFeatureSymbol(string pFeatureName,string pEnumSymbol);
  - ◇ Description: Configure the symbol value of the enumeration attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-39 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pEnumSymbol [IN]	The symbol value of enumeration attribute which is to be configured.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface Class: CardDev, CamDev
- Interface: public int IMV_FG_SetStringFeatureValue(string pFeatureName,string pStrValue);
  - ◇ Description: Configure the string attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-40 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.
pStringValue [IN]	String attribute value which is to be configured.

  - ◇ Reference Routine: **CommPropAccess**.

3.5.3 Execution of Command Attribute

- Interface Class: CardDev, CamDev

- Interface: public int IMV_FG_ExecuteCommandFeature(string pFeatureName);
  - ◇ Descriptions: Execute the command attribute. If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-41 Parameter description

Parameter	Description
pFeatureName [IN]	Attribute name.

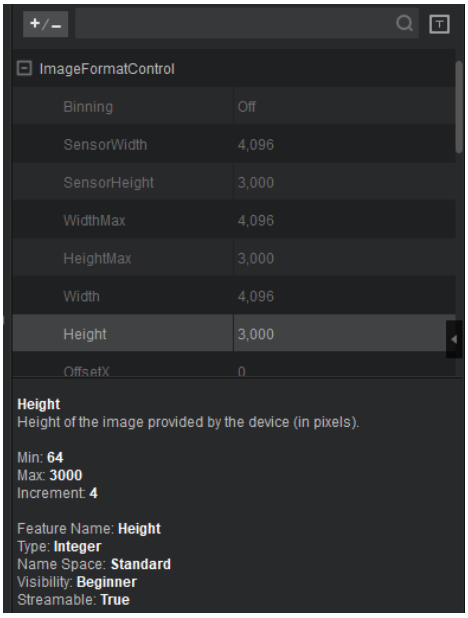
  - ◇ Reference Routine: **CommPropAccess**.

3.5.4 Attribute Search and Usage

User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

3.5.4.1 Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the Height Attribute. The attribute information of it will be displayed in the below. The data type of the Height Attribute is **int** integer as shown in the figure.



User can obtain and modify the found attributes.

```
int ret = IMVFGDfine.IMV_FG_OK;
Int64 heightValue = 0;
Int64 heightMinValue = 0;
Int64 heightMaxValue = 0;
Int64 incrementValue = 0;
```

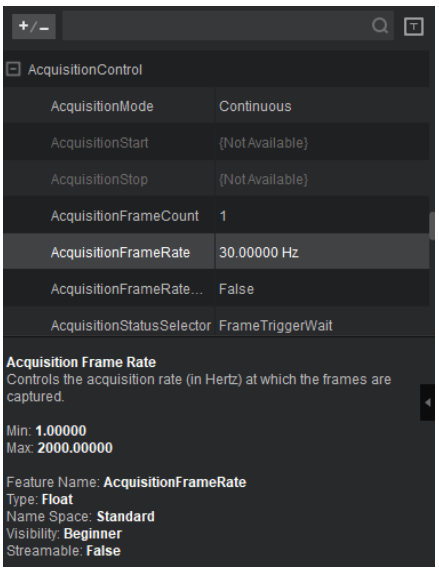
```
CamDev cam=new CamDev();
CardDev card=new CardDev();

  // Obtain attribute value.
   ret = cam.IMV_FG_GetIntFeatureValue("Height",ref heightValue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
   Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
   return ret;
  }

  // Obtain the configurable min value of attribute. If there is no min value, return the
error code.
 ret = cam.IMV_FG_GetIntFeatureMin("Height", ref heightMinValue);
   if(ret != IMVFGDefine.IMV_FG_OK)
   {
    Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
    return ret;
 }
  // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
   ret = cam.IMV_FG_GetIntFeatureMax("Height", ref heightMaxValue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
   Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
   return ret;
 }
  // Obtain the attribute step size. If there is no step-size value, return the error
code.
 ret = cam.IMV_FG_GetIntFeatureInc("Height", ref incrementValue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
   Console.WriteLine($"Get feature increment value failed! ErrorCode[{ret}]");
   return ret;
 }
```

```
    // Configure attribute value.
   ret = cam.IMV_FG_SetIntFeatureValue("Height", heightValue);
    if(ret != IMVFGDefine.IMV_FG_OK)
    {
     Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
     return ret;
   }
```

3.5.4.2 Floating Point Attribute

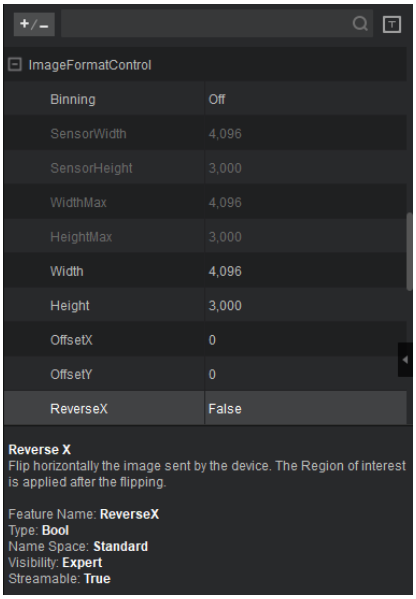



```
    int ret = IMVFGDfine.IMV_FG_OK;
   double framerate = 0;
   double frameratemin = 0;
   double frameratemax = 0;

   // Obtain attribute value.
   ret = cam.IMV_FG_GetDoubleFeatureValue ("AcquisitionFrameRate", ref framerate);
    if(ret != IMVFGDefine.IMV_FG_OK)
    {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
   }
   // Obtain the configurable min value of attribute. If there is no min value, return the
error code.
```

```
    ret = cam.IMV_FG_GetDoubleFeatureMin("AcquisitionFrameRate", ref frameratemin);
      if(ret != IMVFGDefine.IMV_FG_OK)
      {
          Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
          return ret;
    }
      // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = cam.IMV_FG_GetDoubleFeatureMax("AcquisitionFrameRate", ref
frameratemax);
      if(ret != IMVFGDefine.IMV_FG_OK)
      {
          Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
          return ret;
    }
    // Configure attribute value.
    ret = cam.IMV_FG_SetDoubleFeatureValue("AcquisitionFrameRate", framerate);
      if(ret != IMVFGDefine.IMV_FG_OK)
      {
          Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
          return ret;
    }
```

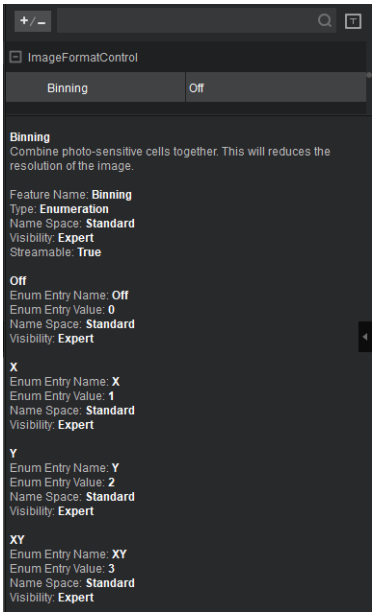
3.5.4.3 Boolean Attribute




```
  int ret = IMVFGDfine.IMV_FG_OK;
bool reversevalue = 0;

// Obtain attribute value.
ret = cam.IMV_FG_GetBoolFeatureValue ("ReverseX", ref reversevalue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
      return ret;
}
  // Configure attribute value.
ret = cam.IMV_FG_SetBoolFeatureValue("ReverseX", reversevalue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
      return ret;
}
```

3.5.4.4 Enumeration Attribute

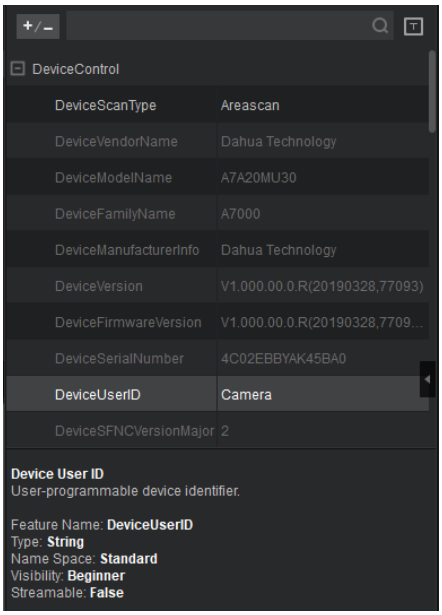



```
    int ret = IMVFGDfine.IMV_FG_OK;
   UInt64 enumValue = 0;
    IMVFGDefine.IMV_FG_String pEnumSymbol = new IMVFGDefine.IMV_FG_String();
    uint pEntryNum = 0;
IMVFGDefine.IMV_FG_EnumEntryList pEnumEntryList = new
IMVFGDefine.IMV_FG_EnumEntryList();

   // Obtain the configurable enumeration value list of attribute.
   ret = cam.IMV_FG_GetEnumFeatureEntrys("Binning", ref pEnumEntryList);
    if(ret != IMVFGDefine.IMV_FG_OK)
    {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
   }
    //Obtain the number of configurable enumeration values of attribute
   ret = cam.IMV_FG_GetEnumFeatureEntryNum("Binning", ref pEntryNum);
    if(ret != IMVFGDefine.IMV_FG_OK)
    {
        Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
        return ret;
   }
```

```
// Obtain the attribute symbol value.
ret = cam.IMV_FG_GetEnumFeatureSymbol("Binning", ref pEnumSymbol);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
      return ret;
}
// Obtain attribute value.
ret = cam.IMV_FG_GetEnumFeatureValue ("Binning", ref enumValue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
      return ret;
}
  // Configure attribute symbol value.
ret = cam.IMV_FG_SetEnumFeatureSymbol("Binning", pEnumSymbol);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
      return ret;
}
  // Configure attribute value.
ret = cam.IMV_FG_SetEnumFeatureValue("Binning", enumValue);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
      return ret;
}
```

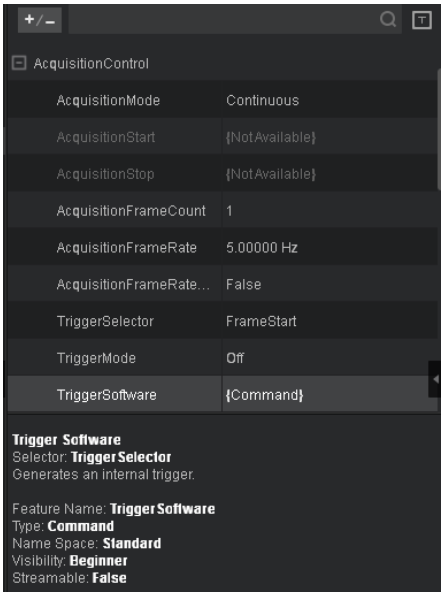
3.5.4.5 String Attribute




```
  int ret = IMVFGDfine.IMV_FG_OK;
string id = "";

// Obtain attribute value.
ret = cam.IMV_FG_GetStringFeatureValue ("DeviceUserID", ref id);
  if(ret != IMVFGDefine.IMV_FG_OK)
  {
      Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
      return ret;
}
  // Configure attribute value.
ret = cam.IMV_FG_SetStringFeatureValue("DeviceUserID", id);
  if(ret != IMVFGDefine.IMV_FG_OK)
{
      Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
      return ret;
}
```

3.5.4.6 Command Attribute




```
  int ret = IMVFGDfine.IMV_FG_OK;
  // Execute the attribute value of command.
ret = cam.IMV_FG_ExecuteCommandFeature ("TriggerSoftware");
  if(ret != IMVFGDefine.IMV_FG_OK)
{
      Console.WriteLine($"Execute Command feature failed! ErrorCode[{ret}]");
      return ret;
}
```

3.6 Image Operation

3.6.1 Video Recording

- Interface Class: CardDev
- Interface: public int IMV_FG_OpenRecord(ref IMVFGDefine.IMV_FG_RecordParam pstRecordParam);
  - ◇ Description: Start recording. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-42 Parameter description

Parameter	Description
pstRecordParam[IN]	Recording parameter struct.

  - ◇ Reference Routine: **Record**.
- Interface Class: CardDev
- Interface: public int IMV_FG_InputOneFrame(ref IMVFGDefine.IMV_FG_RecordFrameInfoParam pstRecordFrameInfoParam);
  - ◇ Description: Record a frame of image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-43 Parameter description

Parameter	Description
pstRecordFrameInfoParam [IN]	Frame information structure for recording.

  - ◇ Reference Routine: **Record.**

3.6.2 Image Operation

- Interface Class: CardDev
- Interface: public int IMV_FG_PixelConvert(ref IMVFGDefine.IMV_FG_PixelConvertParam pstPixelConvertParam);
  - ◇ Description: Pixel format conversion. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-44 Parameter description

Parameter	Description
pstPixelConvertParam [IN][OUT]	Pixel format conversion parameter structure.

- The supported target pixel format: **gvspPixelRGB8/ gvspPixelBGR8/ gvspPixelMono8/ gvspPixelBGRA8**.
- Convert the raw image data into the pixel format required by the user and stored in the specified memory by the caller.
- When pixel format is **YUV411Packed**, the image width must be divisible by 4.
- When pixel format is **YUV422Packed**, the image width must be divisible by 2.
- When pixel format is **YUYVPacked**, the image width must be divisible by 2.
- Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.

◇ Reference Routine: **SoftTrigger, Grab, GrabCallback.**

- Interface Class: CardDev
- Interface: public int IMV_FG_FlipImage(ref IMVFGDefine.IMV_FG_FlipImageParam pstFlipImageParam);

◇ Description: Image flip. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-45 Parameter description

Parameter	Description
pstFlipImageParam [IN][OUT]	Image flip parameter structure.



- The supported pixel format of the vertical and horizontal image flip: gvspPixelRGB8/gvspPixelBGR8/gvspPixelMono8.
- Convert the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**.

- Interface Class: CardDev
- Interface: public int IMV_FG_RotateImage(ref IMVFGDefine.IMV_FG_RotateImageParam pstRotateImageParam);

◇ Description: clockwise rotation of the image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-46 Parameter description

Parameter	Description
pstRotateImageParam[IN][OUT]	Image rotation parameter structure



- The supported image format of the 90/180/270 degrees clockwise rotation: **gvspPixelRGB8** / **gvspPixelBGR8** / **gvspPixelMono8**.
- Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate**

- Interface Class: CardDev
- Interface: public int IMV_FG_SaveImageToFile(ref IMVFGDefine.IMV_FG_SaveImageToFileParampstSaveFileParam);

◇ Description: Save the image into the file. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-47 Parameter description

Parameter	Description
pstSaveFileParam [IN]	Image-saving-into-file parameter structure.



- The image format supported by the interface includes BMP/JPEG/PNG/TIFF.
- The supported max height-width size of JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile.**

4 Image Format of 3rd-party Platform Conversion

4.1 Halcon Object (HObject)

4.1.1 Mono8 (Monochrome Image)

When the image format of camera is Mono8, it constructs the Halcon object from the acquired image data.

In addition, it is assumed here that the **pFrame** is the frame data struct of **IMV_FG_Frame** acquired by SDK

```
HObject ho_image;

HOperatorSet.GenImage1Extern(out ho_image,
                             "byte",
                             (int)pFrame.frameInfo.width,
                             (int)pFrame.frameInfo.height,
                             (HTuple)pData,
                             0);
```

4.1.2 Non-Mono8 (Color Image)

Procedure

Step 1 Convert the acquired image data into the **BGR24** format and consume that the **pData** is the converted BGR data.

Step 2 Construct the **Halcon** image object from the converted **BGR24** image data.

```
HObject ho_image;
HOperatorSet.GenImageInterleaved(out ho_image,
                                 (HTuple)pData,
                                 "bgr",
                                 (int)pFrame.frameInfo.width,
                                 (int)pFrame.frameInfo.height,
                                 0,
                                 "byte",
                                 (int)pFrame.frameInfo.width,
                                 (int)pFrame.frameInfo.height,
                                 0, 0, 8, 0);
```

4.2 MvpImage Object

4.2.1 Mono8 (Monochrome Image)

For formats such as Mono10 and Mono12, refer to **section 3.6.2** for converting the image format to Mono8. When the camera's image format is Mono8, the captured image data can be directly used to construct the object.

It is assumed here that the **pFrame** is the frame struct of **IMV_FG_Frame** acquired by SDK.

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImage1(MvpImgDataType.MVP_IMGDTP_U8,
                      (int) pFrame.frameInfo.width,
                      (int)pFrame.frameInfo.height,
                      pData);
```

4.2.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR24.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK and the **pData** is the BGR data after conversion.

```
HMvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImageInterleaved(pData,
                                EColorSpace.MVP_COLOR_SPACE_BGR,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                MvpImgDataType.MVP_IMGDTP_U8,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                0,
                                -1,
                                0);
```

5 Matched Routine Description

Routine Path:【SDK Installation Directory】\Development\Samples\C#\IMVFG\

Table 5-1 Routine description

Routine Name	Description
BasicDemo	Basic routine.
CommPropAccess	Common attribute routine.
FrameClone	Clone frame.
FrameTriggerCount	Frame trigger statistic.
Grab	Display the manually acquired image.
GrabCallback	Display the callback acquired image.
GrabStrategy	Image acquisition strategy.
ImageConvert	Image transcoding.
ImageFlip	Image flipping.
ImageRotate	Clockwise rotation of the image.
LineTrigger	Hardware trigger to acquire image.
MsgEvents	Message channel.
LoadAndSaveDeviceCfg	Save and load the attribute configuration file of device.
SaveImageToBmp	Save the BIN file and BMP image.
SaveImageToFile	Save image to the PC.
SoftwareTrigger	Software trigger to acquire image.

StreamStatistics	Image acquisition statistics.